

The dot product engine is structured as a 3-stage pipeline. Stage 1 registers the outputs of all 64 multiplications. Each of the 8 engines computes 8 parallel products of the form $M[r][c] \times V[c]$, where each operand is 8 bits and the result is 16 bits. Registering here isolates the multipliers — the deepest combinational blocks in the design — from the rest of the logic, preventing their delay from adding to the adder tree's delay.

Stage 2 registers the output of two adder tree levels computed in the same combinational block. Level 1 reduces 8 products to 4 partial sums by adding neighbouring pairs, requiring 17 bits to avoid overflow. Level 2 reduces those 4 sums to 2, requiring 18 bits. Both levels are collapsed into one pipeline stage because their combined combinational depth fits comfortably within a single clock period. Stage 3 registers the final 2-to-1 addition for each engine, producing the 24-bit output and asserting `valid_out`.

The valid signal is a 1-bit tag that travels through the same register chain as the data — `s1_valid`, `s2_valid`, `valid_out` — guaranteeing it arrives at the output in the exact same cycle as the result, precisely 3 clock cycles after `valid_in`.

With a 3-stage pipeline, the first result appears 3 clock cycles after the input is presented. After that, a new result is produced every single clock cycle (throughput = 1 result per cycle). At 100 MHz, this means 100 million 8×8 matrix-vector products per second. All 8 dot products run simultaneously across 8 parallel engines, meaning 64 multiplications and the full reduction tree execute every clock cycle. A CPU performing the same operation sequentially would need at minimum 64 multiply instructions and 56 add instructions per dot product, totalling 960 operations for all 8 outputs — taking roughly 960 ns at 1 GHz versus 10 ns throughput on this design.

A cpu sums 8 products as $s = p_0 + p_1 + p_2 + \dots + p_7$ places 7 additions in series on the critical path. The adder tree reduces this to $\log_2(8) = 3$ levels, nearly halving the critical path delay and allowing a significantly higher maximum clock frequency. This is the same reason hardware multipliers use Wallace trees rather than ripple-carry chains.

An 8-bit $\times$ 8-bit multiplication produces a maximum value of $255 \times 255 = 65,025$, which requires 16 bits. Summing two 16-bit values requires 17 bits to prevent overflow. Summing two 17-bit values requires 18 bits. The final sum of two 18-bit values requires 19 bits. The output is stored in 24 bits for clean byte-alignment and to provide headroom for future extensions such as accumulating results across multiple tiles or adding a bias term. At no point is any intermediate result truncated, ensuring bit-exact correctness across all input ranges including the maximum stress case of $255 \times 255 \times 8 = 520,200$.